

MEMORY ENGINEERING FOR CODING AGENTS

Building Your Own Memory Layer for Coding Agents

Shashi Jagtap

Founder, Superagentic AI

JUN
04

**The Hidden Layers of AI Systems:
Memory, Retrieval & Search**

MLOps Community London · Thursday, 4 June

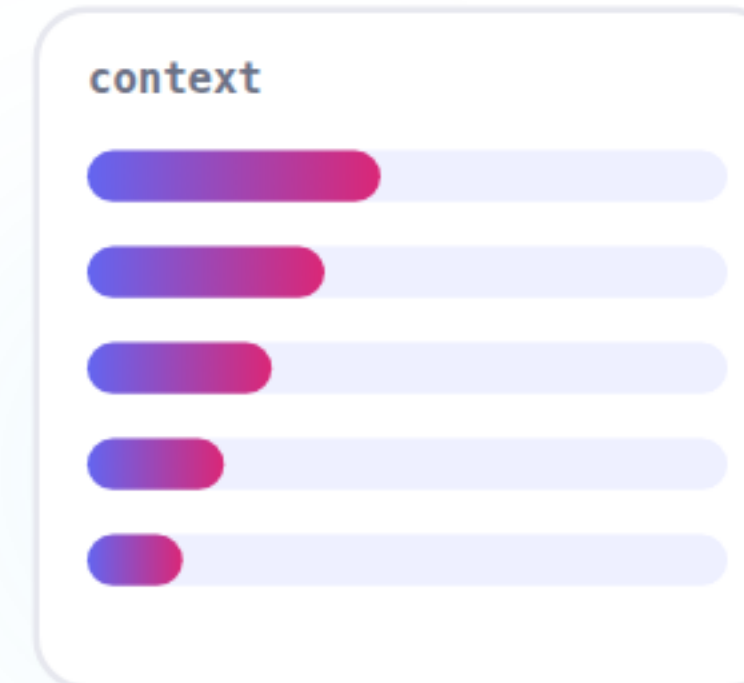
luma.com/9k4ohvyj



• THE PROBLEM

**Working with a coding agent
feels like onboarding a new
developer every single day**

Not a failure of intelligence. A failure of memory.



• THE PROBLEM

Three persistent problems

They repeat across every long running agent session.



01 · SESSIONS

Sessions degrade

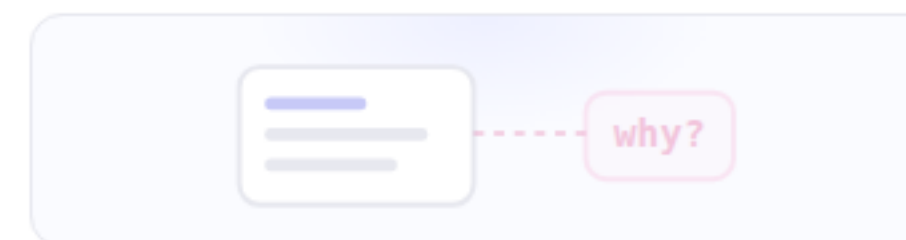
Long context gets compressed, dropped, or forgotten. The reasoning disappears with it.



02 · RETRIEVAL

Retrieval is noisy

Agents find the code, but not the reason behind the code. Code says *what*, never *why*.



03 · INTENT

Intent lives outside code

Requirements, decisions, trade offs, and test mappings vanish between sessions.

• FIRST, A DEFINITION

Memory for a coding agent is its own problem

Most memory tools are built for general assistants. Code is different, so the memory has to be different too.

① GENERAL AGENT MEMORY

- ✓ Remembers **you**: preferences, facts, past chats.
- ✓ Subjective, and it just accumulates over time.
- ✓ A wrong memory is usually harmless.
- ✓ Recency and similarity are mostly enough.

🔗 CODING AGENT MEMORY

- ✓ Remembers the **codebase**: decisions, constraints, conventions, test mappings.
- ✓ Must stay verifiable against code that changes fast.
- ✓ A stale memory ships a bug.
- ✓ Needs exact identifiers *and* the why, not just similarity.

Code is the ground truth. **Memory has to hold the intent, and stay honest as the code moves.**

• THEN IT HAPPENED TO ME

I hit the same wall, in my own work

Using Kiro, my agents generated piles of useful markdown. I had memory. I just could not trust it, find it, or retire it.



• HOW WE WILL APPROACH IT

Let's learn to reason about it, together

 A VOCABULARY

The types of memory, so we stop treating it as one blob.

 A PIPELINE

The stages a real memory layer moves through.

 A SCORECARD

A few questions to reflect on the agent you already run.

• LANDSCAPE

How we got here: a progressive journey

- 1 **Agent files.** `CLAUDE.md`, `AGENTS.md`. Simple, always loaded context you pay for every session.
- 2 **Memory tools.** Agents generate persistent files, progress logs, and steering documents across runs.
- 3 **Skills.** Reusable procedural knowledge: patterns, workflows, and conventions loaded on demand.
- 4 **Agent harnesses.** Structured runtimes like OpenClaw for transparent orchestration and Hermes for compact, self curating memory.
- 5 **Consolidation and dreaming.** Agents review past sessions and rewrite their own memory, like Anthropic's Dreaming, on top of governed memory architectures.

Today, optimization happens at the intersection of **Skills, Harness, and the Memory Layer.**

The current market map

Approach	Examples	Solves	Misses
Always loaded files	CLAUDE.md, AGENTS.md	Simple and readable	Token heavy, static
Scoped instructions	Claude, Copilot rules	Granularity	No lifecycle
Repository retrieval	Cursor, Cody	Code lookup	Weak on intent
Vector memory	Chroma, pgvector	Semantic recall	No structure or freshness
Graph and managed	Mem0, Zep Graphiti	Relationships	Often general purpose
Dedicated frameworks	Supernote, Letta, Engram	MCP and persistence	Varies by tool

Each approach adds value. A vector database is one index, not the layer. **None alone is a complete memory layer.**

• MENTAL MODEL

Memory has types, like human memory

 EPISODIC

What happened

Session transcripts and debugging history.

 SEMANTIC

What is known

Requirements, architecture, and facts.

 PROCEDURAL

How it is done

Coding standards, workflows, conventions. This is Skills.

 WORKING

Right now

The current task and active context.

Typed memory retrieves, filters, ranks, and ages far better than plain text chunks. **Skills are the optimized form of procedural memory.**

- A WAY TO FRAME IT

A memory layer is a pipeline

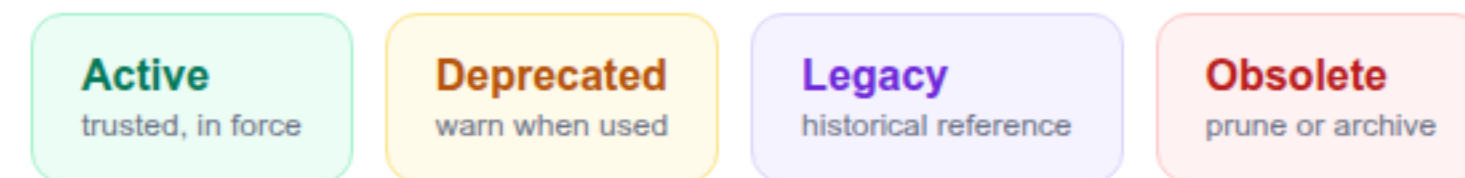


Most teams stop at **Index**. That is just search. The real wins start at **Govern**.

• THE MISSING PIECE

Governance: memory needs a lifecycle

Repositories change fast. A stale decision can be worse than no memory at all. So every memory item carries a **state**.



Plus **pinning** for critical constraints that must never be violated.

• TAKE THIS HOME

Score your agent's memory

Five questions. Every **no** is a gap you now know how to close.

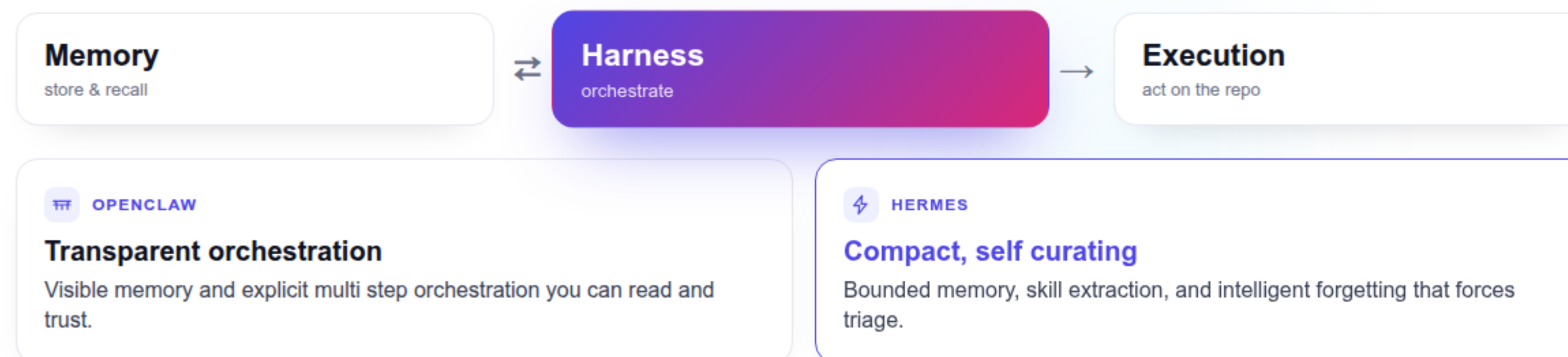
- | | | | |
|---|------------------|--|--------------------------|
| 1 | Typed | Is memory split into episodic, semantic, procedural, working? Or one blob? | ☐ |
| 2 | Retrieved | Hybrid recall: vector plus keyword plus graph? Or similarity only? | ☐ |
| 3 | Governed | Does every item have a lifecycle and pinning? Or does nothing expire? | ☐ |
| 4 | Packed | Do you send the smallest sufficient context? Or ten chunks and hope? | ☐ |
| 5 | Evaluated | Do you measure retrieval like a system that ships? Or never? | ☐ |

Most agents score 1 or 2 out of 5.

• THE BRIDGE

The harness connects memory to execution

The harness manages session handoff, memory read and write, and the optimization loop.



Memory without a capable harness is just storage.

• THE FUTURE

Continual learning is the future of coding agents

Agents must improve over time without catastrophic forgetting. That happens through three layers running as one loop.



Real examples: Anthropic Dreaming, Hermes skill extraction, modular memory systems. **Governed memory is the foundation for safe continual learning.**

• SO, WHAT I DID

Remember my markdown pile? I gave it a memory layer

SpecMem turns those agent generated artifacts into typed, governed memory. I made the four parts concrete. This is just one way to do it, not the way.

 TYPED

Requirement, decision, and test mapping.

 LIFECYCLE

Active, deprecated, legacy, obsolete, with pinning.

 IMPACT GRAPH

Bidirectional links across specs, code, and tests.

 VECTOR STORE

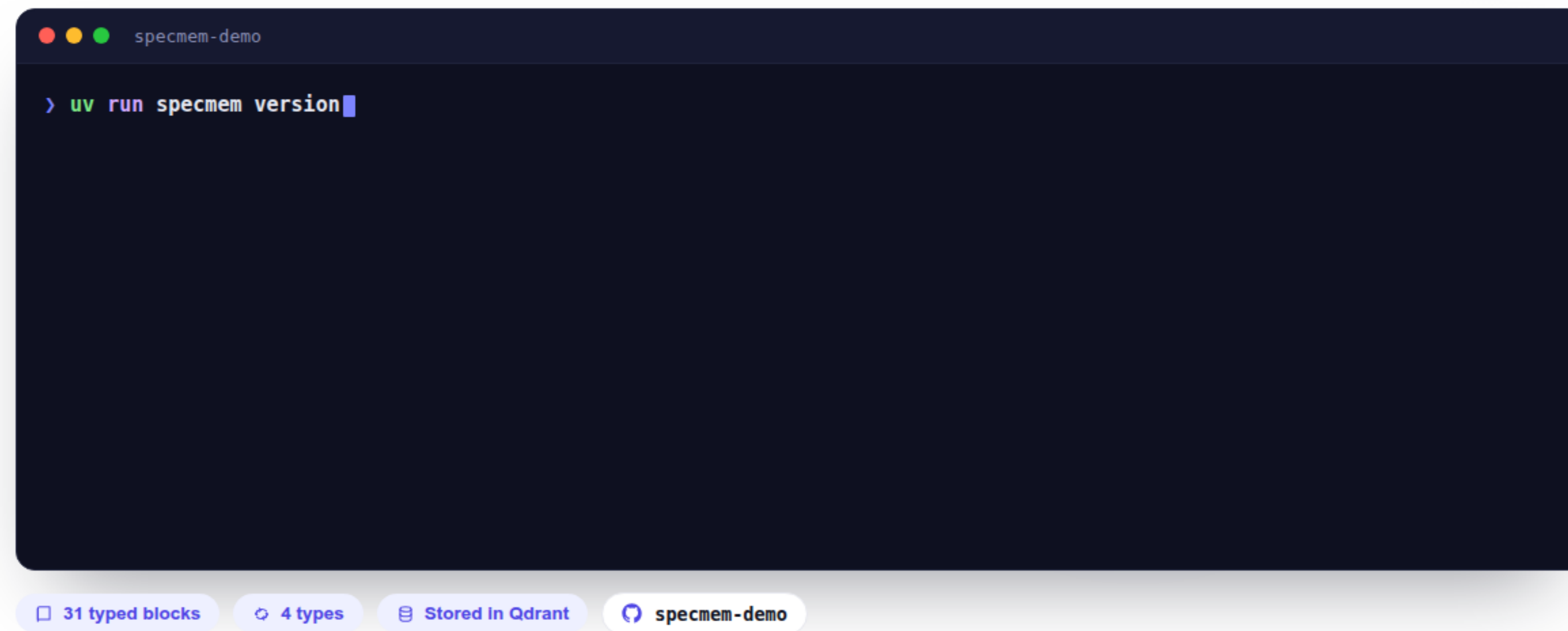
Qdrant, Chroma, or LanceDB as the index.

The point is not the tool. **It is that each of these is a deliberate design choice you can make too.**

 github.com/SuperagenticAI/specmem

• LIVE DEMO

Scan and build: specs become typed memory



• LIVE DEMO

Query: ask in plain English, get the specs that constrain the change

A terminal window titled 'specmem-demo' with a dark background. It shows a command prompt '>' followed by the text 'specmem query "What shou' with a cursor at the end of the line.

```
specmem-demo  
  
> specmem query "What shou
```

RAG retrieves text. **A memory layer returns the specs, decisions, and tests that constrain your change.**

• PROOF

Same edit, same model. One agent remembers.

● agent · no memory

```
> edit src/auth/service.py  
set token TTL → never expires  
tests pass · opened PR
```

Broke a token-expiry rule it never knew existed.

● agent · with a memory layer

```
> edit src/auth/service.py  
⚠ PINNED: tokens must expire after inactivity  
impacted: session.py · test_session_expiry.py  
→ kept TTL, added refresh token
```

Caught before it shipped.

The difference is governed memory.

• DO THIS

What you can start this week

 THIS WEEK

- ✓ Audit your agent's memory: files, logs, rules.
- ✓ Tag lifecycle: Active or Deprecated.
- ✓ Split stable memory from session noise.
- ✓ Try one harness: Hermes or OpenClaw.
- ✓ Script one context bundle.

 THEN

- ✓ Sketch a lightweight memory pipeline.
- ✓ Write 10 "what should I know before changing X?" queries, and evaluate retrieval.
- ✓ Treat Skills as first-class artifacts.

You do not need the perfect tool. Start your memory discipline today.

• THE KEY TAKEAWAY

There is no perfect off the shelf solution

Memory is contextual by definition, and nobody knows your context better than you. Your codebase and team practices are unique.



Solo

Lightweight, local, episodic memory may be all you need.



Team

Shared semantic memory. Decisions and conventions everyone trusts.



Enterprise

Governance, lifecycle, access control, and audit are non negotiable.

A bigger context window helps an agent read more. A better memory layer helps it remember what matters.

Build your own memory layer.

The future belongs to agents with optimized skills, smart harnesses, and governed memory for continual learning.

 [SpecMem](#)

 [SpecMem demo](#)

 [in/shashikantjagtap](#)

 [@Shashikant86](#)

Shashi Jagtap · Superagentic AI · Thank you. Questions?